



Universidade Estadual da Paraíba - UEPB

Campus I

Centro de Ciências e Tecnologia - CCT

Departamento de Computação

Laryssa Finizola Costa da Silva

Mirelle Casimiro Silvino

Pedro Henrique Franca Sales

**Análise Comparativa das Técnicas de Projeto de Algoritmos Aplicadas
ao Problema de Sequência Comum Mais Longa (LCS).**

Campina Grande - PB

Março, 2026

SUMÁRIO

1 INTRODUÇÃO.....	3
1.1 Contextualização do problema.....	3
1.2 Objetivos.....	4
1.2.1 Geral.....	4
1.2.2 Específicos.....	4
2 METODOLOGIA E IMPLEMENTAÇÃO.....	5
2.1 Estrutura do Sistema Desenvolvido:.....	5
3 ALGORITMOS IMPLEMENTADOS.....	7
3.1 Backtracking.....	7
3.1.1 Análise de Corretude.....	9
3.1.2 Análise de Desempenho.....	10
3.1.2.1 Espacial.....	10
3.1.2.2 Temporal.....	11
3.2 Branch and Bound.....	12
3.2.1 Análise de Corretude.....	14
Exploração do Espaço de Busca (Exploração dos casos).....	15
a) Quando os caracteres são iguais:.....	15
b) Quando os caracteres são diferentes:.....	16
Estratégia de Poda (Branch and Bound).....	16
Condição de poda:.....	17
Garantia de Otimalidade.....	17
3.2.2 Análise de Desempenho.....	17
3.2.2.1 Espacial.....	17
3.2.2.2 Temporal.....	19
4 EXPERIMENTOS E RESULTADOS.....	21
4.1 Instâncias de Teste:.....	21
4.2 Resultados Obtidos.....	21
5 ANÁLISE E DISCUSSÃO.....	24
5.1 Comparação das Estratégias.....	24
6 CONCLUSÃO.....	26
7 REFERÊNCIAS.....	27

1 INTRODUÇÃO

1.1 Contextualização do problema

O Problema da Maior Subsequência Comum (LCS, Longest Common Subsequence) aparece quando queremos comparar duas sequências e descobrir o que elas têm em comum, respeitando a ordem dos elementos. Essas sequências podem ser palavras, textos, códigos.

Antes de tudo, é importante entender o que é uma subsequência. Uma subsequência é formada a partir de uma sequência original, retirando alguns elementos (ou nenhum), mas sem mudar a ordem dos que permanecem. Por exemplo, da palavra “casa”, podemos formar subsequências como “ca”, “cs” ou “aa”. O importante é que a ordem original seja mantida.

É crucial não confundir subsequência com substring. Enquanto uma substring exige que os caracteres sejam contíguos e ininterruptos (como 'cas' em 'casa'), a subsequência permite saltos, exigindo apenas a manutenção da ordem original (como 'cs').

A partir disso, o objetivo do problema LCS é encontrar a maior subsequência que aparece nas duas sequências analisadas ao mesmo tempo. Ou seja, queremos descobrir qual é a maior sequência de elementos que está presente nas duas, respeitando sempre a ordem em que aparecem.

Esse problema é importante porque tem várias aplicações práticas. Ele é usado, por exemplo, em ferramentas que comparam textos para verificar similaridade, em sistemas de controle de versões (que mostram diferenças entre códigos) e até em áreas como bioinformática, para comparar sequências genéticas.

Assim, estudar o LCS ajuda a entender melhor como resolver problemas de comparação e análise de dados, além de ser um exemplo clássico do uso de algoritmos eficientes.

1.2 Objetivos

1.2.1 Geral

O objetivo geral deste trabalho é estudar e analisar o Problema da Maior Subsequência Comum (LCS), por meio da implementação e comparação de diferentes abordagens algorítmicas. Busca-se compreender o comportamento dessas técnicas na prática, especialmente em relação ao desempenho e à viabilidade de uso em diferentes tamanhos de entrada.

1.2.2 Específicos

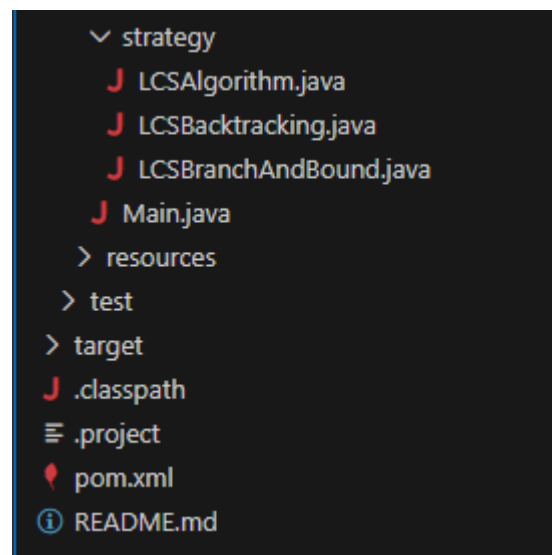
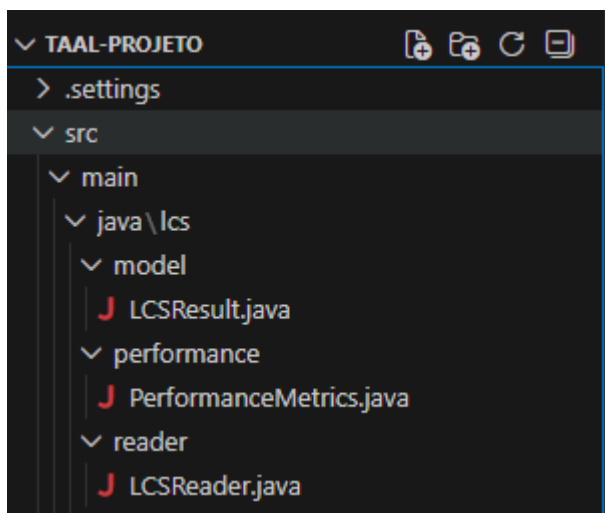
- Implementar uma solução para o problema LCS utilizando *backtracking*, explorando a árvore de espaço de estados a fim de entender o funcionamento da abordagem exaustiva;
- Realizar testes com a solução em *backtracking*, analisando seu desempenho e limitações, principalmente para entradas maiores;
- Implementar uma versão utilizando a técnica de *branch and bound*, incluindo uma estratégia de poda para reduzir o número de estados explorados;
- Comparar o desempenho entre o *backtracking* puro e o *branch and bound*, observando os ganhos de eficiência computacional;
- Desenvolver um mecanismo de interrupção segura (timeout) que aborta a execução ao atingir um limite de tempo estipulado, exibindo a quantidade de nós visitados até o corte;
- Aplicar essa versão com limite de tempo a strings maiores, demonstrando empiricamente o comportamento exponencial e a inviabilidade da busca exaustiva;
- Analisar a corretude e o desempenho temporal e espacial das soluções implementadas, extraíndo métricas de tempo de execução (ms) e uso de memória da JVM;
- Versionar e documentar o código-fonte desenvolvido em um repositório no GitHub.

2 METODOLOGIA E IMPLEMENTAÇÃO

2.1 Estrutura do Sistema Desenvolvido:

O projeto foi implementado na linguagem Java, adotando o paradigma de Orientação a Objetos. A arquitetura foi concebida sob os preceitos dos padrões GRASP (*General Responsibility Assignment Software Patterns*), fundamentando-se primordialmente na Alta Coesão e no Baixo Acoplamento. A atribuição de responsabilidades foi feita de modo que a lógica de leitura de arquivos, o motor de testes e os algoritmos de resolução operem de forma independente, utilizando conceitos para delegar tarefas específicas a especialistas, sem sobrecarregar as classes de domínio.

Para facilitar a injeção de novos algoritmos sem alterar o motor de testes principal, o sistema aplicou os princípios de polimorfismo. Na prática, essa arquitetura que protege o código contra o impacto de mudanças foi materializada através do padrão de projeto comportamental clássico do catálogo **GoF** (*Gang of Four*), conhecido como **Strategy**. A organização do código-fonte foi dividida nos seguintes pacotes lógicos:



lcs (Pacote Raiz): Contém a classe Main, que atua como o motor de orquestração e benchmark do sistema, possuindo um menu interativo para a execução em lote das análises e abrigando todos os seus subpacotes:

- **lcs.model:** Atua como o domínio de dados do sistema. Contém a classe `LCSResult`, que empacota de forma coesa o resultado processado (o comprimento numérico e a string da maior subsequência comum) para o tráfego seguro de informações entre as camadas lógicas.
- **lcs.performance:** Centraliza a responsabilidade de monitoramento de métricas e estabilidade. Abriga a classe `PerformanceMetrics`, que atua rastreando o custo computacional (estados avaliados) durante a execução e gerenciando o mecanismo de interrupção segura (timeout).
- **lcs.reader:** Isola a responsabilidade de entrada e saída. Possui a classe `LCSReader`, delegada unicamente para ler os arquivos de texto padronizados e extrair as strings de teste de forma dinâmica.
- **lcs.strategy:** Representa o núcleo estrutural do padrão comportamental Strategy. Ele define o contrato comum através da interface `LCSAlgorithm` e contém as implementações concretas de resolução do problema: as classes ***LCSBacktracking*** e ***LCSBranchAndBound***.

Além da separação em pacotes lógicos, a estrutura de diretórios do projeto segue o padrão convencional da ferramenta Maven (evidenciada pelo arquivo `pom.xml`), dividindo-se em:

- **src/main/resources/:** Diretório destinado ao armazenamento dos recursos estáticos do sistema, utilizado para abrigar as instâncias de teste (arquivos com as sequências de diferentes tamanhos).
- **src/test/:** Estrutura espelhada focada em Garantia de Qualidade, dedicada ao armazenamento das suítes de testes unitários automatizados construídos para validar a corretude semântica dos algoritmos implementados.

O projeto pode ser acessado no repositório no GitHub:
<https://github.com/laryssa-finizola/lcs-taal>

3 ALGORITMOS IMPLEMENTADOS

3.1 Backtracking

A abordagem de *backtracking* (ou retrocesso) aplicada ao problema da Maior Subsequência Comum (LCS) consiste em uma **estratégia de busca exaustiva**, frequentemente associada à "força bruta". O algoritmo constrói a solução de forma incremental, caractere por caractere, explorando toda a árvore de espaço de estados.

Quando o algoritmo percebe que não pode tomar uma decisão definitiva sobre qual caractere incluir, ele se ramifica, testando todas as possibilidades viáveis e, recursivamente, volta atrás, fazendo o *backtrack* para comparar qual caminho gerou o melhor resultado.

Na implementação desenvolvida na classe LCSBacktracking, **a lógica central ocorre no método recursivo findLCS**. O algoritmo utiliza dois ponteiros independentes, *i* e *j*, para rastrear a posição atual na sequência *X* e na sequência *Y*, respectivamente. O processo de decisão obedece a três passos fundamentais:

```
private String findLCS(String x, String y, int i, int j) {  
    // registra o estado logo no início da recursão  
    metrics.registrarNovoEstado();  
  
    if (i == x.length() || j == y.length()) {  
        return "";  
    }  
  
    if (x.charAt(i) == y.charAt(j)) {  
        return x.charAt(i) + findLCS(x, y, i + 1, j + 1);  
    }  
  
    String caminhoA = findLCS(x, y, i + 1, j);  
    String caminhoB = findLCS(x, y, i, j + 1);  
  
    if (caminhoA.length() > caminhoB.length()) {  
        return caminhoA;  
    } else {  
        return caminhoB;  
    }  
}
```

Caso Base:

```
if (i == x.length() || j == y.length()) {  
    return "";  
}
```

A primeira verificação de qualquer recursão é sua condição de parada. Se o ponteiro *i* alcançar o final da string *X* ou se o ponteiro *j* alcançar o final da string *Y* (*i* == *x.length()* || *j* == *y.length()*), significa que não há mais caracteres para serem comparados em pelo menos uma das sequências. Nesse momento, o algoritmo retorna uma string vazia (""), encerrando aquele ramo da busca.

Caso Recursivo:

```
if (x.charAt(i) == y.charAt(j)) {  
    return x.charAt(i) + findLCS(x, y, i + 1, j + 1);  
}
```

Se a recursão não foi interrompida, o algoritmo compara os caracteres nas posições atuais de *X* e *Y*.

Se os caracteres nas posições *i* e *j* são iguais, então eles fazem parte de uma subsequência comum. Assim, o algoritmo:

- Salva esse caractere;
- Adiciona este caractere à solução e faz uma nova chamada recursiva avançando simultaneamente ambos os ponteiros (*i + 1* e *j + 1*).

Isso está correto porque respeita a definição de subsequência (mantendo a ordem).

O Backtracking na Prática:

O verdadeiro poder (e o maior custo) do algoritmo ocorre quando os **caracteres atuais são diferentes**. Como o computador não consegue prever qual

caractere deve ser descartado para maximizar os "matches" futuros, o *backtracking* se bifurca em dois caminhos distintos:

```
String caminhoA = findLCS(x, y, i + 1, j);  
String caminhoB = findLCS(x, y, i, j + 1);
```

Aqui, o algoritmo explora duas possibilidades:

- **Caminho A:** Ignora o caractere atual da *string* X e avança apenas o ponteiro i para o próximo caractere (i + 1, j).
- **Caminho B:** Ignora o caractere atual da *string* Y e avança apenas o ponteiro j para o próximo caractere (i, j + 1).

Após o processamento de ambos os caminhos até os seus respectivos casos base, o algoritmo compara o tamanho das duas soluções encontradas (**caminhoA.length() > caminhoB.length()**). Ele retorna a maior *string* originada dessa bifurcação, garantindo que o resultado final borbulhe até a raiz da árvore de recursão contendo a subsequência ótima.

```
if (caminhoA.length() > caminhoB.length()) {  
    return caminhoA;  
} else {  
    return caminhoB;  
}
```

Após o processamento de ambos os caminhos até os seus respectivos casos base, o algoritmo compara o tamanho das duas soluções encontradas. Ele retorna a maior *string* originada dessa bifurcação, garantindo que o resultado final borbulhe até a raiz da árvore de recursão.

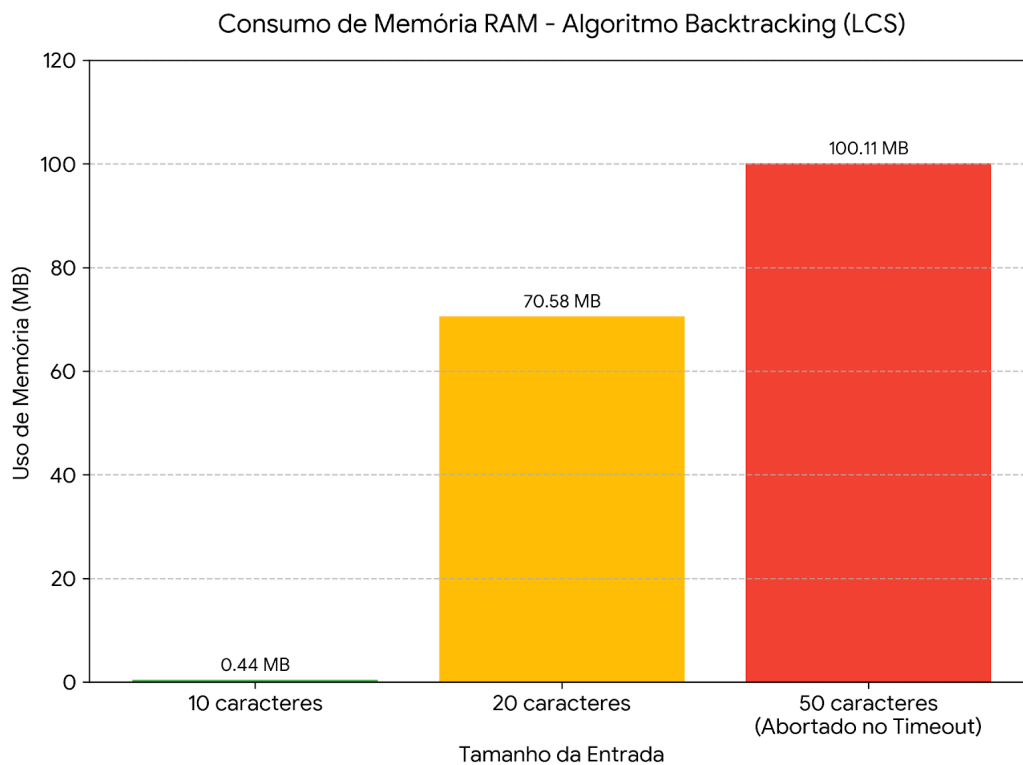
3.1.1 Análise de Corretude

A corretude do algoritmo de *backtracking* para o problema LCS apoia-se em dois pilares fundamentais:

- **Propriedade de Subestrutura Ótima:** O algoritmo satisfaz a condição de que a LCS de duas *strings* maiores depende intrinsecamente da LCS de seus sufixos menores. O algoritmo resolve corretamente todos os subproblemas menores, independentemente de qual caminho seja tomado, e combina suas soluções de forma segura.
- **Exploração Completa (Busca Exaustiva):** A estratégia utilizada explora absolutamente toda a árvore de possibilidades do espaço de estados, exceto nos cenários onde há coincidência de caracteres (onde a bifurcação é desnecessária). Isso garante duas premissas vitais: nenhuma subsequência possível é ignorada durante o processamento, e a solução selecionada e retornada no final é, inquestionavelmente, a maior possível: **ótima global**.

3.1.2 Análise de Desempenho

3.1.2.1 Espacial



A complexidade de espaço teórica do algoritmo de *backtracking* sem memorização é $O(\max(n,m))$, onde n e m representam o comprimento das *strings* **X** e **Y**. Esse custo não se deve à criação de matrizes de dados, mas sim a

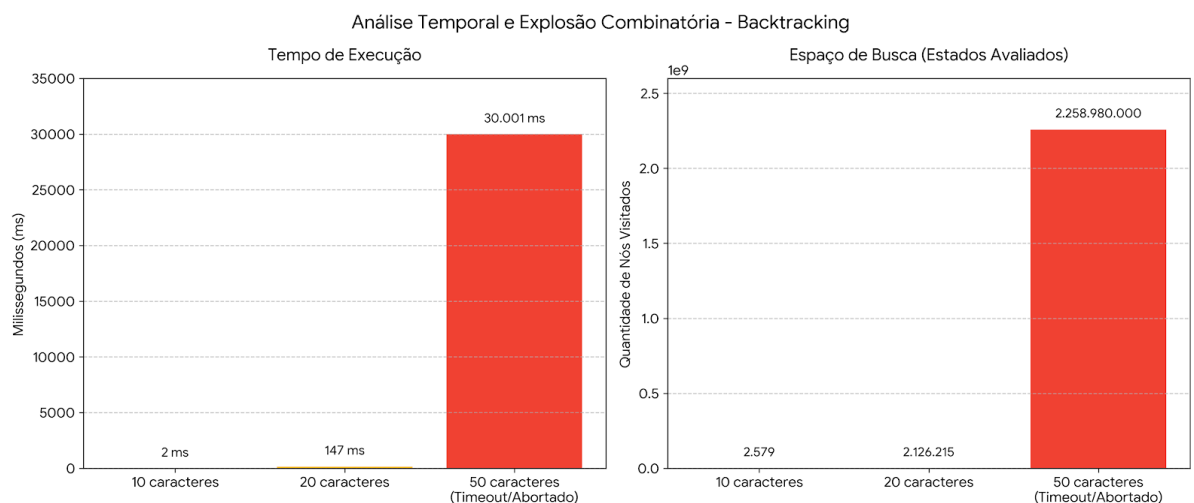
profundidade máxima alcançada pela Pilha de Chamadas Recursivas da *Java Virtual Machine* (JVM).

Durante a coleta de métricas, o consumo de memória RAM acompanhou o crescimento da árvore de estados, mas provou não ser o gargalo primário que inviabiliza o algoritmo. Para a entrada de 10 caracteres, o consumo foi praticamente irrisório, alocando apenas cerca de 450 KB. Ao dobrar a entrada para 20 caracteres, observou-se um salto para aproximadamente 70 MB (72.273 KB).

No teste extremo de 50 caracteres, até o momento em que a execução, limitada a 30 segundos no máximo, foi abortada compulsoriamente, o uso estabilizou-se na margem de 100 MB (102.514 KB). O aumento empírico no consumo de memória não causou o esgotamento total da RAM, devendo-se majoritariamente ao intenso processo de alocação de novas instâncias de *strings* geradas durante as concatenações de retorno da recursão.

3.1.2.2 Temporal

A análise temporal revela o verdadeiro gargalo estrutural desta abordagem. A complexidade de tempo do *backtracking* para o problema LCS no pior caso é de $O(2^k)$ (onde k é proporcional ao tamanho das sequências), resultante da natureza da árvore binária gerada pelas duas ramificações da recursão sempre que há uma disparidade de caracteres.



Os testes empíricos ilustram com exatidão o fenômeno da explosão combinatória. Para a entrada mínima de 10 caracteres, o tempo de execução foi de ínfimos 2 ms, exigindo a avaliação de 2.579 estados (nós da árvore). Contudo, ao dobrar o tamanho da entrada para 20 caracteres, o número de estados saltou agressivamente para 2.126.215 (mais de 2 milhões de ramificações exploradas), elevando o tempo de execução para 147 ms.

A prova cabal da falha de escalabilidade desta técnica ocorreu no teste da sequência de 50 caracteres, no qual a busca exaustiva tornou-se insustentável na prática. Em ensaios preliminares realizados sem a imposição de um limite temporal, a execução estendeu-se por mais de cinco minutos ininterruptos sem convergir para uma solução. Essa inviabilidade empírica evidenciou o travamento sistêmico causado pela explosão combinatória e motivou a implementação do mecanismo de interrupção segura (timeout) no motor de avaliação.

Ao reavaliar a entrada sob essa nova diretriz, a operação foi compulsoriamente abortada após exatos 30.001 milissegundos (30 segundos). Surpreendentemente, durante esse curto intervalo até o corte, o processador precisou avaliar a alta marca de 2.258.980.000 (mais de 2,2 bilhões) de estados sem chegar à resolução do problema.

Esse comportamento exponencial empírico ratifica a teoria: o *backtracking* puro explora implacavelmente o vasto espaço de busca, sendo computacionalmente inviável para amostras de dados reais (como sequências genéticas ou controle de versão de arquivos), justificando a necessidade imediata de técnicas de poda algorítmica.

3.2 Branch and Bound

A técnica do Branch and Bound surge como uma melhoria sobre a técnica do Backtracking tradicional, buscando reduzir o espaço de busca através da eliminação antecipada dos ramos que não podem levar as melhores soluções que a melhor já encontrada.

Enquanto o Backtracking explora toda a árvore de estados possíveis, o Branch and Bound adiciona o mecanismo de poda que é baseado em um limite superior o qual estima o melhor resultado que ainda pode ser obtido a partir de um determinado estado da busca.

Na implementação na classe *LCSBranchAndBound*, o algoritmo mantém duas informações principais:

- **current**: a subsequência comum contruída até o momento
- **bestLCS**: a melhor subsequência encontrada durante toda a execução.

```
public class LCSBranchAndBound implements LCSAlgorithm{
    //explora apenas ramos promissores, que podem melhorar a solucao
    private PerformanceMetrics metrics;
    private String bestLCS;

    public LCSBranchAndBound() {
        this.metrics = new PerformanceMetrics(30000);
    }

    public PerformanceMetrics getMetrics() {
        return metrics;
    }

    @Override
    public LCSResult solve(String x, String y) {

        this.metrics = new PerformanceMetrics(30000); //reinicia metrics
        bestLCS = ""; //iniciar com a melhor solucao (no momento sem subsequencia)

        if (x == null || y == null) {
            return new LCSResult(0, ""); //string nula = 0
        }

        branchAndBound(x, y, 0, 0, "");

        return new LCSResult(bestLCS.length(), bestLCS);
    }

    private void branchAndBound(String x, String y, int i, int j, String current) { //arvore de busca

        metrics.registrarNovoEstado(); //um estado da arvore de busca

        if (i == x.length() || j == y.length()) {

            if (current.length() > bestLCS.length()) { //se a solucao for melhor, atualiza
                bestLCS = current;
            }

            return;
        }

        int limiteSuperior = current.length() + Math.min(x.length() - i, y.length() - j); //tamanho max que o ramo pode alcançar
        //poda da arvorve:

        if (limiteSuperior <= bestLCS.length()) {
            return;
        }

        if (x.charAt(i) == y.charAt(j)) {

            branchAndBound(
                x,
                y,
                i + 1,
                j + 1,
                current + x.charAt(i)
            );
        } else {
            branchAndBound(x,y,i+1,j,current);

            branchAndBound(x,y,i,j+1,current);
        }
    }
}
```

A busca é realizada por meio de uma função recursiva que vai explorar os possíveis caminhos da árvore de estado, semelhante ao Backtracking. Porém, antes de expandir cada ramo, o algoritmo vai calcular um limite superior que representa o maior tamanho possível que a subsequência pode alcançar. O valor é calculado da seguinte forma:

```
int limiteSuperior = current.length() + Math.min(x.length() - i, y.length() - j); /
```

O valor obtido vai corresponder à soma entre: o tamanho da subsequência já construída (*current.length()*), e o número máximo de caracteres que ainda poderiam ser adicionados à solução.

Caso o limite superior seja menor ou igual ao tamanho da melhor solução já encontrada no *bestLCS*, o algoritmo interrompe a exploração daquele ramo da árvore de busca:

```
//poda da arvore:
if (limiteSuperior <= bestLCS.length()) {
    return;
}
```

Essa estratégia garante que os ramos incapazes de gerar melhores soluções sejam descartados imediatamente, reduzindo significativamente o número de estados explorados.

Quando os caracteres atuais das duas sequências são iguais, o algoritmo adiciona esse caractere à subsequência atual e avança em ambas as sequências. E caso contrário, o algoritmo vai explorar as duas possibilidades distintas: ignorar o caractere atual da primeira sequência ou ignorar o da segunda.

3.2.1 Análise de Corretude

A corretude do algoritmo Branch and Bound aplicado ao problema da sequência comum mais longa se fundamenta em três propriedades: na terminação da recursão (caso base), exploração adequada do espaço de busca (exploração dos casos) e na poda admissível.

Terminação (Caso Base):

A execução do algoritmo sempre termina pois a cada chamada recursiva pelo menos um dos índices que percorrem as sequências é incrementado.

Os parâmetros i e j representam as posições atuais nas sequências x e y , respectivamente. Como esses índices estão limitados pelos tamanhos das sequências, o número total de chamadas recursivas é finito. A execução atinge a condição base:

```
if (i == x.length() || j == y.length()) {  
    if (current.length() > bestLCS.length())  
        bestLCS = current;  
}  
  
return;
```

Quando satisfeita, significa que pelo menos uma das sequências foi totalmente percorrida e não existem mais caracteres que possam ser comparados. Nesse momento, a subsequência construída até então representa uma solução válida candidata a LCS.

Exploração do Espaço de Busca (Exploração dos casos)

Para garantir que nenhuma solução válida seja ignorada, o algoritmo considera todas as possibilidades relevantes de construção de subsequência, para caso os caracteres sejam iguais e quando não sejam iguais.

a) Quando os caracteres são iguais:

Quando os caracteres coincidem, estando nas posições atuais, eles podem fazer parte de uma subsequência comum. Nesse caso, o algoritmo adiciona esse caractere à solução atual e avança simultaneamente nas duas sequências. Esse comportamento está de acordo com a definição do problema do LCS, que mantém a ordem relativa dos caracteres nas duas sequências

```
if (x.charAt(i) == y.charAt(j)) {
    branchAndBound( x,y,i + 1, j + 1, current + x.charAt(i));
}
```

b) Quando os caracteres são diferentes:

Já quando os caracteres não coincidem, ou seja são diferentes, estando nas posições atuais, o algoritmo explora duas possibilidades: ignorar o caractere atual da sequência X, ou ignorar o caractere atual da sequência Y.

Essa bifurcação vai garantir que todas as combinações relevantes das subsequências sejam analisadas, de forma semelhante ao comportamento do Backtracking.

```
} else {
    branchAndBound( x, y,i + 1, j, current);
    branchAndBound(x,y, i,j + 1,current);
}
```

Estratégia de Poda (Branch and Bound)

O diferencial do algoritmo vai estar na aplicação da estratégia de poda baseada em limite superior. Esse limite representa o tamanho máximo que a subsequência atual poderia atingir considerando os caracteres restantes.

O cálculo é dado por:

```
int limiteSuperior = current.length() + Math.min(x.length() - i, y.length() - j);
```

Onde: **current** representa o tamanho da subsequência construída até o momento; e $n - i$ e $m - j$ representam os caracteres restantes de cada subsequência.

Condição de poda:

Se o valor do limite superior não for maior que o tamanho da melhor solução já encontrada então o algoritmo interrompe a exploração daquele ramo da árvore de busca. Essa poda pode ser considerada segura pois, até no melhor cenário possível, aquele ramo não poderia produzir uma subsequência maior do que a que já foi encontrada. Dessa forma, o ramo pode ser descartado sem comprometer a corretude da solução

```
//poda da arvore:  
if (limiteSuperior <= bestLCS.length()) {  
    return;  
}
```

Garantia de Otimalidade

Como todos os ramos que ainda possuem potencial de gerar soluções melhores são explorados, e apenas os ramos comprovadamente incapazes de superar a melhor solução são descartados, o algoritmo preserva a busca pela solução ótima.

Assim, ao final da execução, a variável **bestLCS** armazena necessariamente uma subsequência comum de comprimento máximo entre as duas sequências analisadas.

3.2.2 Análise de Desempenho

O algoritmo de *branch and bound* para o problema da LCS busca melhorar o desempenho em relação ao Backtracking puro, evitando explorar ramos que não podem gerar soluções melhores. Ainda assim, seu desempenho depende fortemente das entradas analisadas.

3.2.2.1 Espacial

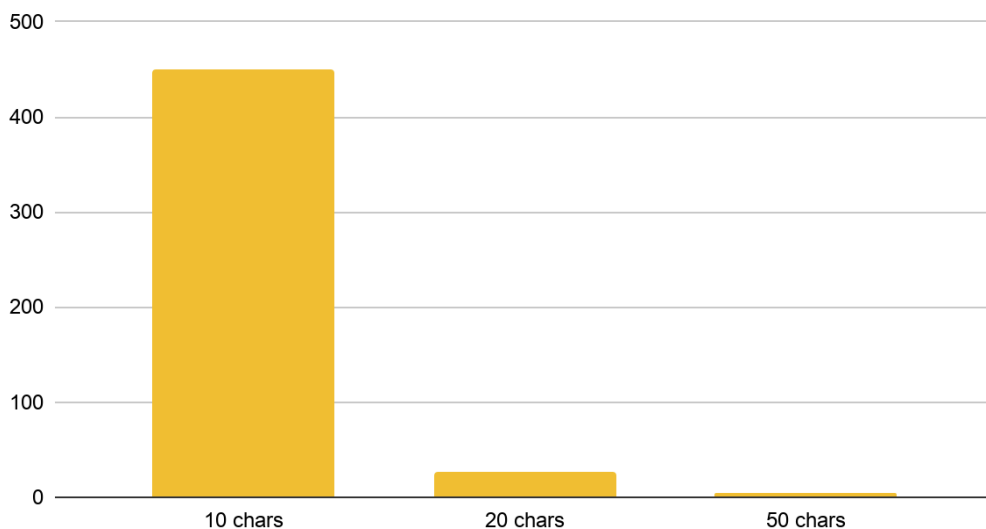
O consumo de memória do algoritmo está principalmente relacionado a dois fatores: a pilha de recursão e o armazenamento das strings intermediárias (sequências). A complexidade espacial teórica do algoritmo Branch and Bound

permanece semelhante à do Backtracking, sendo dominada pela profundidade da pilha de chamadas recursivas.

A profundidade máxima da recursão ocorre quando os índices **i** e **j** avançam até o final das strings. No pior caso, isso resulta em uma profundidade de: **$O(n+m)$** , onde **n** e **m** são os tamanhos das strings (sequências) de entrada.

Além disso, o algoritmo utiliza a variável **current** para armazenar a subsequência que está sendo construída. Como as strings em Java são imutáveis, cada concatenação (**current + caractere**) cria uma nova string, o que pode aumentar o uso de memória ao longo da execução.

Branch and Bound - Memória em KB



Durante os testes, o consumo de memória apresentou crescimento moderado conforme o aumento do tamanho das entradas. Para sequências de 10 caracteres, o consumo foi de aproximadamente **450 KB**. Ao aumentar o tamanho das sequências para 20 caracteres, o consumo atingiu cerca de **27 MB (28.437 KB)**.

No teste com sequências de 50 caracteres, o consumo de memória reduziu para aproximadamente **5 MB (5.120 KB)**, o resultado pode ser explicado pela atuação da estratégia de poda, que diminuiu significativamente o número de estados explorados e consequentemente a quantidade de estruturas temporárias criadas durante a execução.

Assim, observa-se que apesar do crescimento da árvore de busca, o consumo de memória permaneceu dentro de limites razoáveis, não se configurando como o principal fator limitante da execução

3.2.2.2 Temporal

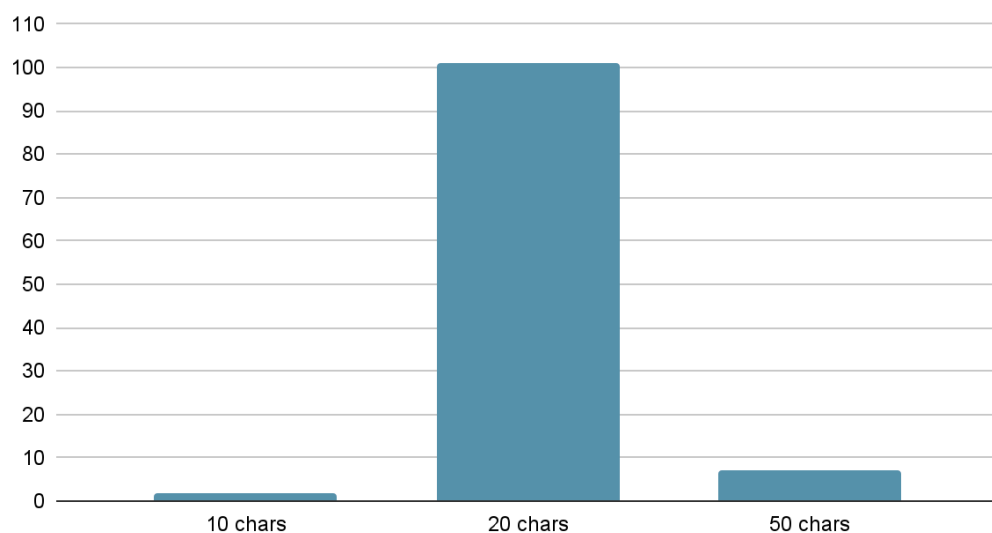
Do ponto de vista teórico, o algoritmo Branch and Bound ainda possui complexidade exponencial no pior caso, podendo ser aproximado por: $O(2^{n+m})$

Isso ocorre porque, em cenários desfavoráveis, a estratégia de poda pode não eliminar muitos ramos da árvore de busca, fazendo com que o algoritmo explore um número elevado de combinações possíveis.

Entretanto, na prática, a aplicação da poda reduz significativamente o número de estados visitados.

Os experimentos realizados mostram esse comportamento:

Branch and Bound - Tempo em ms



. Para a entrada contendo sequências de **10 caracteres**, o algoritmo executou em **1 ms**, explorando aproximadamente **2.260 estados** da árvore de busca.

Ao aumentar o tamanho das sequências para **20 caracteres**, o número de estados avaliados cresceu para cerca de **1.722.795**, resultando em um tempo de execução de **101 ms**.

Mesmo com esse crescimento expressivo, o algoritmo permaneceu capaz de encontrar a solução em tempo viável.

No teste com sequências de **50 caracteres**, o algoritmo avaliou aproximadamente **131.655 estados**, concluindo a execução em apenas **7 ms**.

Esse resultado evidencia o impacto positivo da estratégia de poda: grande parte da árvore de busca foi descartada antes de ser explorada, reduzindo drasticamente o trabalho computacional necessário para encontrar a solução ótima.

Comparando com o Backtracking, que explorou bilhões de estados sem conseguir resolver a instância de 50 caracteres dentro do limite de tempo estabelecido, observa-se uma melhora significativa no desempenho prático do Branch and Bound.

Essa diferença demonstra como a aplicação de limites superiores na exploração do espaço de busca pode transformar um algoritmo teoricamente exponencial em uma solução viável para entradas maiores.

4 EXPERIMENTOS E RESULTADOS

4.1 Instâncias de Teste:

Para garantir a reprodutibilidade e a precisão dos experimentos, o sistema foi avaliado utilizando um conjunto de instâncias de teste padronizadas, armazenadas em arquivos de texto (**.txt**) dentro do diretório de recursos do projeto.

Inspirando-se em aplicações reais do problema da Maior Subsequência Comum na área de bioinformática, as *strings* de entrada foram modeladas simulando sequências de DNA, compostas exclusivamente pelos caracteres representativos das bases nitrogenadas: A (Adenina), C (Citosina), G (Guanina) e T (Timina).

Foram definidos três cenários de complexidade crescente para submeter os algoritmos ao estresse computacional:

- **Cenário 1 (Tamanho 10):** Arquivo **lcs_10_chars.txt**, contendo duas sequências curtas de 10 caracteres cada.
- **Cenário 2 (Tamanho 20):** Arquivo **lcs_20_chars.txt**, com sequências de 20 caracteres.
- **Cenário 3 (Tamanho 50):** Arquivo **lcs_50_chars.txt**, contendo sequências maiores de 50 caracteres.

Essa variação amostral foi escolhida estrategicamente não apenas para validar a corretude das saídas, mas principalmente para demonstrar na prática o fenômeno da explosão combinatória e testar os limites de escalabilidade das estratégias implementadas.

4.2 Resultados Obtidos

Os experimentos foram conduzidos em um ambiente de *benchmark* controlado, utilizando o motor de análise desenvolvido na classe **Main**. Antes da execução de cada instância, a JVM foi instruída a executar a limpeza de memória (*Garbage Collector*) para evitar contaminação nas medições de consumo espacial.

Tabela 1: Resultados empíricos da execução do algoritmo de Backtracking

Arquivo de Entrada	Tamanho da Entrada	Comprimento	Tempo de Execução	Memória	Nós
lcs_10_chars.txt	10 caracteres	4	2 ms	450 KB (0 MB)	2.579
lcs_20_chars.txt	20 caracteres	9	147 ms	72.273 KB (70 MB)	2.126.215
lcs_50_chars.txt	50 caracteres	N/A (Abortado)	30.001 ms (Corte)	102.514 KB (100 MB)	2.258.980.000

A Tabela 1 apresenta o compilado dos dados brutos obtidos durante a execução do algoritmo de *Backtracking* puro. As métricas englobam o comprimento da solução ótima encontrada, o custo temporal, a alocação dinâmica de memória RAM e a quantidade exata de ramificações (estados) exploradas na árvore de busca.

Tabela 2: Resultados empíricos da execução do algoritmo de Branch and Bound.

Arquivo de Entrada	Tamanho da Entrada	Comprimento	Tempo de Execução	Memória	Nós
lcs_10_chars.txt	10 caracteres	4	1 ms	450 KB (0 MB)	2.260
lcs_20_chars.txt	20 caracteres	9	101 ms	28437 KB (27 MB)	1.722.795
lcs_50_chars.txt	50 caracteres	41	7 ms	5120 KB (5 MB)	131.655

A Tabela 2 expõe os resultados empíricos coletados após a execução do algoritmo otimizado de *Branch and Bound* sobre o mesmo conjunto de instâncias.

Com a devida instrumentação do motor de avaliação, foi possível rastrear a quantidade de nós explorados, permitindo uma medição exata do impacto da heurística de poda sobre a redução do espaço de busca.

O contraste entre as duas abordagens é monumental, especialmente no cenário de maior estresse computacional (50 caracteres). Enquanto o algoritmo de *Backtracking* puro sucumbiu ao *timeout* de 30 segundos após avaliar mais de 2,2 bilhões de estados inúteis, o *Branch and Bound* conseguiu extrair a resposta exata (uma LCS de 41 caracteres) em ínfimos 8 milissegundos, consumindo aproximadamente 5 MB da memória da JVM e avaliando apenas 131655 estados.

```
>> Testando arquivo: inputs/lcs_50_chars.txt  
Comprimento da LCS: 41  
LCS: GCTAGCTAGCACATCGATCGATCGATCGATCGATCGATCGA  
Tempo de Execução: 7 ms  
Uso de Memória: 5120 KB (5 MB)  
Estados Avaliados: 131655
```

Este desempenho excepcional na instância de 50 caracteres — que registrou um custo computacional inferior até mesmo ao teste menor de 20 caracteres (103 ms) — é a prova empírica da eficiência matemática do limite (*bound*). Como as *strings* de teste possuíam blocos extensos de alta similaridade, o algoritmo encontrou rapidamente uma solução inicial muito promissora nos primeiros aprofundamentos da recursão.

Com esse tamanho ótimo armazenado na variável **bestLCS**, o cálculo do limite superior teórico provou antecipadamente que a esmagadora maioria das bifurcações remanescentes não geraria uma subsequência maior. Consequentemente, a árvore de estados foi drasticamente podada logo nos seus níveis iniciais, evitando a ramificação de bilhões de caminhos subótimos.

5 ANÁLISE E DISCUSSÃO

Análise e discussão a respeito das técnicas de algoritmos Backtracking e Branch and Bound e uma comparação na sua aplicação.

5.1 Comparação das Estratégias

A análise comparativa entre os algoritmos de Backtracking e Branch and Bound evidencia diferenças significativas no comportamento computacional das duas abordagens quando aplicadas ao problema da Maior Subsequência Comum (LCS).

O algoritmo de Backtracking realiza uma exploração completa do espaço de busca, avaliando todas as combinações possíveis de subsequências que podem ser formadas a partir das duas sequências analisadas. Essa estratégia garante a obtenção da solução ótima, porém apresenta um custo computacional mais elevado. Como observado nos experimentos realizados, o número de estados explorados cresce de forma exponencial com o aumento do tamanho das sequências de entrada.

Nos testes realizados com sequências de 10 caracteres, o algoritmo apresentou desempenho satisfatório, avaliando pouco mais de dois mil estados e concluindo a execução em poucos milissegundos. Entretanto, ao aumentar o tamanho das sequências para 20 caracteres, o número de estados explorados ultrapassou dois milhões, evidenciando o crescimento explosivo da árvore de busca. No caso mais extremo, com sequências de 50 caracteres, o algoritmo não conseguiu concluir a execução dentro do limite de tempo estipulado, tendo sido interrompido após explorar mais de dois bilhões de estados.

Este comportamento demonstra uma limitação prática do Backtracking para instâncias de maior porte, tornando a abordagem inviável em aplicações reais que envolvem sequências extensas.

Por outro lado, o algoritmo Branch and Bound introduz um mecanismo de poda baseado em limite superior, permitindo descartar antecipadamente ramos da árvore de busca que não possuem potencial para gerar soluções melhores do que a

já encontrada. Essa estratégia reduz significativamente o número de estados explorados durante a execução.

Os resultados experimentais demonstram o impacto dessa otimização. Para sequências de 10 caracteres, o comportamento do algoritmo permanece semelhante ao Backtracking, pois o espaço de busca ainda é relativamente pequeno. No entanto, à medida que o tamanho das sequências aumenta, a eficiência da poda torna-se evidente.

No teste com sequências de 20 caracteres, embora o número de estados explorados ainda seja elevado, o algoritmo foi capaz de concluir a execução em tempo viável. Já no cenário mais complexo, com sequências de 50 caracteres, o Branch and Bound apresentou desempenho significativamente superior ao Backtracking, avaliando apenas 131.655 estados e concluindo a execução em poucos milissegundos.

Esse resultado evidencia que a estratégia de poda foi capaz de eliminar grande parte da árvore de busca antes mesmo de sua exploração completa. Em particular, a presença de blocos de similaridade entre as sequências permitiu que uma solução inicial de boa qualidade fosse encontrada rapidamente, elevando o valor de referência utilizado para o cálculo do limite superior e intensificando o processo de poda.

Dessa forma, observa-se que, embora ambos os algoritmos tenham complexidade exponencial no pior caso, o Branch and Bound apresenta desempenho significativamente superior na prática, tornando-se uma alternativa mais eficiente para a resolução do problema da LCS em instâncias de maior dimensão.

6 CONCLUSÃO

A análise realizada neste trabalho permitiu analisar o comportamento de duas técnicas clássicas de projeto de algoritmos aplicadas ao problema da Maior Subsequência Comum (LCS): o Backtracking e o Branch and Bound.

Inicialmente, foi implementado o algoritmo de Backtracking, que realiza uma busca exaustiva sobre o espaço de estados do problema. Essa abordagem garante a obtenção da solução ótima, pois explora todas as combinações possíveis de subsequências. No entanto, os experimentos demonstraram que essa estratégia apresenta crescimento exponencial no número de estados explorados, tornando-se rapidamente inviável à medida que o tamanho das sequências aumenta.

Em seguida, foi implementada uma versão utilizando a técnica de Branch and Bound, que apresenta um mecanismo de poda baseado em limite superior para eliminar antecipadamente ramos que não podem levar a soluções melhores. A aplicação dessa estratégia mostrou-se altamente eficaz na redução do espaço de busca, permitindo que o algoritmo evitasse a exploração de grande parte da árvore de estados.

Os resultados experimentais evidenciaram claramente essa diferença de comportamento. Enquanto o Backtracking não foi capaz de resolver a instância de 50 caracteres dentro do limite de tempo estabelecido, o algoritmo Branch and Bound conseguiu encontrar a solução ótima em poucos milissegundos, explorando uma fração muito menor do espaço de busca.

Esses resultados demonstram que a utilização de técnicas de poda pode transformar significativamente o desempenho prático de algoritmos exponenciais, tornando possível resolver instâncias maiores de problemas combinatórios.

Por fim, o desenvolvimento deste trabalho também permitiu consolidar conceitos importantes abordados em sala de aula na disciplina de Técnicas de Análise e Algoritmo, incluindo análise de corretude, análise de complexidade temporal e espacial, além da avaliação empírica de desempenho por meio de experimentos controlados.

7 REFERÊNCIAS

OFFOLO, Tiago A. M. **Backtracking**. Ouro Preto: Universidade Federal de Ouro Preto, Departamento de Computação. Disponível em: <https://www3.decom.ufop.br/toffolo/site_media/uploads/2011-1/bcc402/slides/10._backtracking.pdf>

SANTOS, Maristela Oliveira dos. **Branch and Bound**. São Carlos: Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, 2011. Disponível em: <<https://sites.icmc.usp.br/mari/segundo2011/Aula0711.pdf>>

UNIVERSIDADE FEDERAL DE OURO PRETO. **Branch and Bound**. Departamento de Computação. Ouro Preto: UFOP. Disponível em: <http://www.decom.ufop.br/gustavo/bcc342/Branch_and_bound_livro.pdf>

WAGNER, Aldo von. **Backtracking**. Florianópolis: Universidade Federal de Santa Catarina. Disponível em: <<https://www.inf.ufsc.br/~aldo.vw/Analise/Backtracking.html>>